

AI Voice Interview Copilot

Human Conversation, Follow-Up & Speech Accuracy Audit

Scope: Can this system understand what a real human *meant* — through pauses, fillers, corrections, imperfect pronunciation and STT errors?

Date: 2026-08-23 • **Build:** working tree at commit `7a375cf` + uncommitted changes to 9 files • **Mode:** read-only — **no application file was modified**

Evidence base: 11,598 logged events from 241 real session files (1,441 live-call transcripts, 465 real answers) · 132-case offline gate matrix · 52 synthesised speech files through the real VAD + live Groq Whisper · 70 degraded-audio STT runs · 20 live multi-turn conversations against the real answer model · 55 timing trials

1. Executive Summary

The system is **fast and, on a clean transcript, genuinely good**. End-of-speech to first spoken word is ~800 ms at p50 in real sessions. A clean four-turn follow-up chain (*What is RAG?* → *Why use it?* → *What are the disadvantages?* → *Have you used it?*) was answered correctly, in context, with the style changing appropriately at each turn. Whisper with the configured vocabulary prompt held the term "RAG" intact through all ten audio-degradation conditions tested.

The system is **fragile everywhere a human is not fluent**. Every failure found in this audit falls into one of three families:

1. The pipeline decides a question is finished before the human has finished it.

A mid-question pause splits one question into two answers, or throws the second half away. Measured: "Can you explain..." + "RAG?" produced **two separate answers at every pause length from 0.2 s to 4.0 s** — the split is unconditional, not a tuning problem. "Will you give me the..." + "idea about your recent projects." merged correctly below a 1.6 s pause and **silently discarded the real question** above 2.0 s. Both patterns appear verbatim in the production logs.

2. A self-correction is never applied. "Tell me about your Flask project. Actually, I mean my Django project." answered **Flask**, at every pause length tested, and never answered Django. In a real session, "I said the `HTTPS methods`." — the interviewer correcting a mishearing — was not even evaluated by the gate.

3. A mis-transcribed technical term is answered literally, and the error then propagates. "What is Rack?" (RAG misheard — 21 real occurrences in the logs) was answered as **Ruby Rack middleware**, and the follow-up "Why would you use it?" continued about Nginx and Puma — with a résumé and job description that both name RAG explicitly. In production, "you work with a doctor" (Docker) produced an invented story about integrating HL7 feeds for a health-tech client.

Underneath all three sits one structural gap: **there is no layer anywhere in the pipeline whose job is to repair the transcript**. The VAD segments audio, STT produces text, a quality filter deletes text it distrusts, a rule gate decides *whether* and *which part* to answer — and then the raw string goes to the LLM. The only correction mechanism in the entire system is one sentence in the system prompt asking the model to "silently infer the most plausible real term." That sentence works when the mis-transcription is phonetically obvious (*fast API* → FastAPI, *deployment pack* → deployment part) and fails when the mis-transcription is itself a real word with its own technical meaning (*Rack*, *doctor*, *red is*).

A fourth issue is not about understanding but is arguably the most dangerous in an interview: **the answers invent facts about the candidate**. Of 28 quantified claims across 33 generated answers, **28 appear nowhere in the supplied résumé or job description** — "cut ticket handling time by 30 %", "documentation set grew past 200 GB", "reduced code-review cycles by about 30 percent". The same project was given two different, incompatible numbers in two answers of the same session. The system prompt forbids exactly this and calls it "the one hard line."

Headline numbers

Measure	Result	Source
End-of-speech → LLM call starts (real sessions)	p50 182 ms	465 real answers, <code>since_last_fragment_ms</code>
LLM call → first visible word (real sessions)	p50 598 ms	latest session, n=42
Rule-gate accuracy over 132 realistic utterances	115 / 132 (87 %)	Test A
— of which: incomplete/trailing-off questions	1 / 9	Test A
One question split into two answers by a pause	2 of 5 patterns, at every gap	Test E, 55 trials
Self-corrections applied	0 / 3 tested	Tests A2, D, E
Mis-transcribed term recovered by the LLM	3 / 6 tested	Test D (live model)
Quantified claims in answers not present in the résumé/JD	28 / 28	Test D, 33 answers
Single-part questions answered with bullets (prompt forbids)	12 / 18	Test D
Answers over the prompt's own 90-word ceiling	18 / 33 (55 %)	Test D; p50 = 107 words
Real transcripts ending "." vs "?"	67.5 % vs 23.0 %	1,441 live transcripts
Answer capacity before provider throttling	≈ 4–5 questions/min	~1,400–1,700 tok/question vs 8,000 TPM

What is already working and must not be broken

- **Latency.** 182 ms from end-of-speech to the LLM call is excellent and was clearly hard-won. Every recommendation in this report must be evaluated against it.
- **Hallucination filtering.** "Thank you.", "Um...", "So," on near-silence were all correctly dropped in testing; the three-signal filter (voiced duration + decoder confidence + word shape) does its job.
- **Clean follow-up chains.** Four consecutive pronoun follow-ups resolved correctly with visibly different answer shapes per turn.
- **Vocabulary biasing.** "RAG" survived noise down to 6 dB SNR, telephone band-limiting, 30 % level loss and clipping — *because it is in the STT prompt*. "Flask", which is not, broke in 8 of the same 10 conditions.
- **Observability.** The durable JSONL log is what made this audit possible at all. Very few systems this size can be audited from their own logs.

2. Current Architecture (as built, not as assumed)

Interviewer speaks (remote party, heard through the speakers) | ▼ `client/test_loopback_stream.py` – WASAPI `loopback` capture, not a microphone. | Reads in 100 ms gulps, re-slices to 20 ms / 16 kHz mono PCM16, drops the OLDEST | buffered audio when the socket falls behind. The overlay is a separate process. ▼ `/ws/audio` (app/routes/audio_ws.py) — one WebSocket per session | ▼ `SpeechSegmenter` (app/services/vad.py) – energy VAD, adaptive noise floor | • 1,000 ms calibration: reports NO speech at all while measuring the line | • trigger = $\max(150, \text{noise_floor} \times 2.5)$; release at $0.6 \times \text{trigger}$ (hysteresis) | • onset 3 frames; 300 ms pre-roll; segment ends after 420 ms of silence | • a segment needs ≥ 320 ms of VOICED audio or it is thrown away un-transcribed | • force-cut at 8 s with 200 ms carry-over → same utterance_id, is_final=False ▼ produces `AudioSegment(pcm, speech_seconds, utterance_id, index, is_final)` | ↳ `speech_active` edge → `pipeline.set_speech_active()` (the gate's only "they are talking now" signal) ↳ segment → `_transcription_dispatcher` (≤ 3 STT calls in flight) → `_result_consumer` (delivers in spoken order) | ▼ **STT** – Groq `whisper-large-v3-turbo`, 12 s timeout, prompt-biased with a fixed | technical vocabulary + JD + résumé, truncated to 850 UTF-8 bytes. | Fallback to OpenAI `Whisper` is ENABLED in config but has NO KEY → no fallback exists. ▼ `TranscriptionResult(text, confidence, confidence_known, no_speech_prob)` | ▼ **Transcript quality filter** (`transcript_quality.py`) – DROPS the text if: | looping repetition • $\text{lexical_word_ratio} = 0$ • $\text{ratio} < 0.5$ • stock silence phrase with | < 1 s voiced audio • $\text{confidence} < 0.35$ • $\text{no_speech_prob} \geq 0.6$ ▼ ▼ `QuestionAnswerPipeline.submit_transcript()` → per-session buffer of fragments | ▼ **Decision loop** (`_process_after_debounce`) – sleeps 30 ms (settled) or 180 ms, then: | ① **Tier-1 rule gate** `_classify_question()`:

normalise → small-talk → follow-up phrase → | split into clauses on [.!]? → score each clause (interview-intent fuzzy match, | credible "?", question prefix, question phrase, "X vs Y", bare tech keyword) → | pick the LAST strong clause → grow the focus outward over neighbouring real sentences | ② **Tier-2 LLM classifier** – ONLY when tier-1 says no_question_signal and the | utterance is closed. openai/gpt-oss-20b, 1.2 s cap, replies ANSWER / WAIT / IGNORE | ③ **Wait policy** _should_keep_waiting(): hold, fire, or discard ▼ ▼ **Context assembly** – résumé (2,000 ch) + JD (1,500 ch) + notes (500 ch) + last 3 Q&A turns | Follow-ups only: _resolve_followup() rewrites the question to name the previous Q&A | keyword_match only: _resolve_keyword_mention() replaces the text with "talk about <term>" | garbled recognised intent only: _resolve_interview_intent() states what was meant ▼ ▼ **LLM** – Groq openai/gpt-oss-120b, temp 0.3, max 500 tokens (shared with hidden reasoning) ▼ streamed to /ws/answers → overlay, and appended to the durable JSONL log

Stage-by-stage: what can go wrong, and can the next stage fix it?

Stage	Receives → Produces	What can go wrong	Can a later stage recover it?
Audio capture	Speaker output → 20 ms PCM frames	WASAPI ring-buffer overflow corrupts a read; queue-full drops the oldest audio; 100 ms gulp latency; only the remote party is heard	No. Lost audio is indistinguishable downstream from silence. The drop is counted but nothing acts on it.
VAD	Frames → segments	Deaf for the first 1,000 ms of a connection; quiet speech below $\max(150, \text{noise} \times 2.5)$ never produces a segment; a >420 ms mid-sentence pause ends the utterance; <320 ms of voice is discarded	No. A segment that is never emitted leaves no trace except a sub_threshold_runs counter logged at session close.
STT	Segment → text + confidence	Technical terms mis-transcribed; short fragments score low confidence; language switches to Devanagari on Hinglish; invented text on near-silence; no fallback recognizer configured	Partly. Only by the LLM, and only when the wrong word is not itself a real technical word.
Quality filter	Text → keep / drop	Deletes a short, correct second half of a question (measured: "FAST API." dropped at confidence < 0.35)	No. Deletion is final; only a transcript_dropped event is emitted.
Question gate	Buffer → answer / hold / discard + focus	Fires on half a question; truncates the question via clause splitting or intent matching; misses follow-ups that end in "."; accepts noise with a spurious "?"	No. The focus string is what the LLM sees; everything outside it is gone.
Context	Session store → résumé/JD/history	History is written only after an answer <i>completes</i> ; a cancelled answer leaves no history for the next follow-up. Only the last 3 turns are sent.	—
LLM	Messages → streamed answer	Answers a mis-transcription literally; echoes the wrong term into the spoken answer; invents metrics; exceeds its own format budget; empty answer when reasoning eats the token budget	No stage exists after this one.

3. Test Methodology

ID	Test	What it exercises	Scale / notes
LOG	Production log mining	Real Indian-English interviewer speech through the whole pipeline	11,598 events, 241 files, 7 days; 1,441 transcripts; 1,019 gate decisions; 465 answers
A	Offline gate matrix	_classify_question over 132 realistic utterances across 10 categories, incl. verbatim production transcripts	Deterministic, no network. 0.57 ms/case
A2	Mechanism isolation	Punctuation sensitivity, acronym clause-splitting, and the real _should_keep_waiting policy at 4 time points	Deterministic

ID	Test	What it exercises	Scale / notes
B/C	Audio → VAD → live STT → filter → gate	52 synthesised utterances wrapped in 1.6 s lead-in / 1.2 s tail of 28-RMS comfort noise, fed 20 ms at a time through the real SpeechSegmenter , then the real Groq Whisper service	Windows SAPI, two US-English voices, rate −5/0/+5. Limitation: no Indian-accent TTS is available on this machine — see §17
D	Live multi-turn conversations	The real QuestionAnswerPipeline driven exactly as audio_ws.py drives it (speech-active edges, utterance ids, STT delay), with the real Groq answer model, a realistic résumé and JD	20 conversations, 33 answers
E	Pause-length sweep	5 two-part questions × 11 pause lengths (0.2–4.0 s), stubbed LLM for determinism	55 trials
F	Degraded-audio STT	7 technical questions × 10 channel conditions (noise 20/12/6 dB SNR, 300–3400 Hz telephone band, 30 % level, clipping, combinations)	70 live STT calls

Evidence labelling used throughout: **VERIFIED** reproduced from code or a test run, with the observation quoted **OBSERVED** seen in real runtime, root cause not fully isolated **INFERRED** strong hypothesis from code reading, not directly triggered **RECOMMENDATION** proposed change, not implemented.

4. Human Speech Test Matrix — results by category

Test A, 132 utterances. "Correct" means the gate reached the decision a competent human listener would: answer a real question, hold an unfinished one, ignore noise.

Category	Correct	What the failures were
Technical word misheard	39 / 39	The <i>gate</i> handles every mis-transcription tested. The failure is downstream, in the LLM (§9).
Mispronunciation variants	4 / 4	rag / rack / R.A.G. / RAG all answered. One truncated the question — see §8.
Scenario questions	7 / 7	Setup-only sentences correctly held; setup+question correctly answered together.
Multi-part questions	4 / 4	All parts retained in the focus.
Informal / Hinglish grammar	10 / 10	"How RAG works?", "Why you used RAG?", "Docker you have used or not?", "Aap ne Docker use kiya hai?" all accepted.
Fillers	7 / 8	Only the indirect ask failed: "You know, I wanted to ask about RAG." → <code>no_question_signal</code> .
Follow-ups	21 / 24	"Go on.", "Anything else?", "Okay and then?" not recognised.
Noise / non-questions	13 / 17	Answered: "Okay, would I?", "Corrections, Frappe.", "So it's Docker.", "I don't know what I'm doing." — all verbatim from real sessions.
Self-correction	9 / 10	The gate is not the main problem here; the <i>timing</i> is (§12).
Incomplete questions	1 / 9	The worst category. Eight of nine trailing-off fragments are classified answerable and fire after ~1.7 s.
Total	115 / 132 (87 %)	

5. VAD Findings

5.1 Utterance grouping does not cover natural pauses VERIFIED

The VAD's `utterance_id` mechanism is described in the code as the signal that tells the gate "more of this sentence is coming." It only survives a **force-cut at 8 s of continuous speech**. Any pause longer than `segment_end_silence_ms` (420 ms) closes the utterance and the next word starts a new one with `is_final=True`.

Measured across every multi-part utterance in Tests B/C — twelve of them — **every single segment came back as its own utterance with `index_in_utterance = 0` and `is_final = True`**:

```
pause_a_gap600ms    segs=2  utterance_ids=[1, 2]  both is_final=True
pause_b_gap1200ms   segs=2  utterance_ids=[1, 2]  both is_final=True
scenario_long        segs=6  utterance_ids=[1,2,3,4,5,6] all is_final=True
correct_sql          segs=4  utterance_ids=[1,2,3,4]  all is_final=True
```

Consequence: `awaiting_more_of_utterance` is **false for every ordinary paused question**, so the branch in `_should_keep_waiting` that is supposed to make waiting free and certain never runs. The gate is left with the timing guess it was designed to replace.

In the production logs the grouping *does* fire — 41 of 119 utterances (34.5 %) carried more than one transcript — but examining them shows it firing on **continuous audio longer than 8 s**, which in practice was music and background chatter, not questions. One real utterance glued a song lyric to a genuine question: ["You belong right in this moment. I'm falling fast...", "But suddenly CPU reaches 90 to 95%."].

5.2 A filler word becomes its own segment VERIFIED

"Um," / "So," / "Basically," before the question are separated by enough silence to close a segment:

```

filler_um      → 2 segments: ["Um..."(dropped as silence_hallucination), "Can you explain RAG?"]
filler_basically → 3 segments: ["So,"(dropped), "Basically,", "What is RAG?"]
filler_like    → 3 segments: ["What is RAG?", "Like?", "In simple terms."]

```

The filter correctly deletes the stock ones. The cost is extra STT calls (3 requests for one 4-word question) and extra decision cycles, not wrong answers — except in `filler_like`, where the gate's focus selection then dropped "In simple terms." (\$14).

5.3 Deafness windows that leave no trace INFERRED VERIFIED in code

- **First 1,000 ms of every connection** (`vad_calibration_ms`): the detector returns `False` for every frame regardless of content. If the capture client is (re)started while the interviewer is already talking, that opening is lost with no log line.
- **Speech under the trigger** (`max(150, noise_floor × 2.5)`): produces no segment, no STT call, no event. The only trace is a `sub_threshold_runs` counter written at session close. This is the correct instrumentation but it is post-hoc — nothing tells the operator during the interview.
- **Segments with < 320 ms of voiced audio** are discarded. This is what makes the hallucination filter work, and it is also what will drop a genuinely terse "Why?" spoken quickly.

6. STT Findings

6.1 On clean audio, accuracy is high VERIFIED

44 synthesised utterances: 39 transcribed exactly. Errors were `Django` → "D'Ango", `Kubernetes` → "Kubernets", and formatting normalisations (`"GPT four"` → "GPT-4", `"CI CD"` → "CI-CD", `"R A G"` → "RAG") which are improvements, not errors.

6.2 Degradation breaks the terms that are *not* in the vocabulary prompt VERIFIED

Test F, 7 questions × 10 channel conditions. The result is unambiguous:

Term	In STT prompt?	Conditions broken	What Whisper produced
RAG	yes	0 / 10	"RAG" in all ten, including 6 dB SNR + telephone band + 35 % level
Docker	yes	0 / 10	"Docker" in all ten
Kubernetes	yes	9 / 10	"Kubernets" ×7, "Kibernets", "Kavir Nets"
Redis	yes	4 / 10	"Reedy's"
Django	yes	4 / 10	"Gango" ×3, "D'Ango" (on the <i>clean</i> file)
FastAPI	yes	5 / 10	"FAST API" — split into two words
Flask	no	8 / 10	"Vlesk" ×2, "Blask" ×3, "Blast", "Black" ×3

Meaning inversion. In 3 of 10 conditions the whole question flipped polarity: "Why did you choose FastAPI over Flask?" → "Why **didn't** you choose FAST API over Black?". The gate accepted it as a credible question and the LLM would answer the negation. No stage detects this.

6.3 The résumé never reaches the recognizer once a JD is loaded VERIFIED

`_build_stt_prompt()` concatenates `stt_prompt` (433 bytes) + `JD[:500]` + `résumé[:300]` and truncates the result to 850 UTF-8 bytes. With a 500-character JD the total is 1,114 bytes, so **264 bytes are cut from the tail — and the tail is the entire résumé**. Measured:

```

base+JD(500)+resume(300) = 1114 bytes → truncated to 850 bytes
JD text reached the recognizer:      True
resume text reached the recognizer:  False
discarded: "UDAY UGALE -- AI / Full-Stack Engineer. Mannlowe Information Services, Pune.

```

```
Built a RAG chatbot ... Chroma vector store; FSM-driven onboarding agent on
FastAPI; Django REST, PostgreSQL, Redis, Celery; React, Redux."
```

The code comment says this biasing "is what fixes misheard jargon like React 19 → reactivity." For the résumé half, on any session with a job description, it is not running at all.

6.4 39 common interview terms are absent from the vocabulary VERIFIED

Flask, MySQL, Celery, Pinecone, Nginx, GitHub, Linux, Pandas, NumPy, Terraform, Jenkins, Golang, JWT, OAuth, Redux, Tailwind, Pydantic, uvicorn, pytest, Airflow, Spark, Snowflake, Kotlin, Swift, Angular, Vue, Spring Boot, .NET, Hugging Face, Ollama, vLLM, Milvus, Qdrant, Weaviate, FAISS, BERT, LoRA, fine-tuning, transformer.

Flask is the control case: absent from the list, and broken in 8 of 10 conditions while RAG and Docker held in 10 of 10.

6.5 Short fragments score low confidence and can be deleted VERIFIED

Measured confidence against voiced duration on the same recogniser:

voiced	conf	text
0.42	0.45	'RAG.'
0.42	0.46	'RAG.'
0.46	0.54	'Sorry.'
0.82	0.46	'In your project?'
0.90	0.54	'Can you explain?'
—	<0.35	'FAST API.' ← DELETED by the filter (min_confidence 0.35)

In the "Why did you choose... [0.8 s] ...FastAPI?" run, the second half was deleted. The gate then saw "Why did you choose?", judged it complete (it ends in "?"), fired immediately, and the live model answered "*I chose that stack because FastAPI lets me spin up lightweight async endpoints...*" — a lucky guess from the résumé, not an answer to the question asked.

6.6 No fallback recognizer despite the configuration saying otherwise VERIFIED

STT_FALLBACK_ENABLED=true with OPENAI_API_KEY empty. The warning fired on every audit run. Any STT failure or 12-second timeout loses that segment outright.

7. Mispronunciation Findings

Four renderings of the same request, all through live STT and the real gate:

Spoken	Transcript	Gate	Question sent to the LLM
"Tell me about your recent RAG project."	identical	interview_intent	full sentence
"...recent R A G project"	"...recent RAG project."	interview_intent	full sentence — Whisper normalised the spelling-out
"...recent R.A.G. project." (as text)	—	interview_intent	"Tell me about your recent R.A.G." — "project." was dropped (§8)
"...recent rack project."	—	interview_intent	full sentence, but the LLM then decides what "rack" means (§9)

Conclusion: mispronunciation is not a gate problem. The gate answers all four. Whether the *meaning* survives is decided entirely by the LLM, with no help from any earlier stage.

8. Technical Word Findings — clause shredding by acronyms

VERIFIED `_CLAUSE_BOUNDARY = re.compile(r"(?<=[.?!])\s+")` splits on any full stop followed by whitespace. `Node.js` and `Next.js` are safe because there is no space after the dot. A spelled-out acronym is not:

Utterance	Clauses produced	Sent to the LLM
Explain the A.P.I. design you used.	["Explain the A.P.I.", "design you used."]	"Explain the A.P.I." — lost "design you used"
Have you deployed on A.W.S. before?	["Have you deployed on A.W.S.", "before?"]	"Have you deployed on A.W.S." — lost "before?"
Do you write S.Q.L. queries daily?	["Do you write S.Q.L.", "queries daily?"]	"Do you write S.Q.L." — lost "queries daily?"
Tell me about your recent R.A.G. project.	["...recent R.A.G.", "project."]	lost "project."
The server runs at 99.9. percent uptime. How do you monitor it?	3 clauses	"How do you monitor it?" — scenario setup discarded
What is an L.L.M. and how does it work?	2 clauses	whole sentence (the "?" clause won and growth reached back)
Okay. Okay. Dr. What is the REST API?	4 clauses	"What is the REST API?" — junk correctly stripped

The rule that decides whether a neighbouring clause is kept is `_is_content_clause: ≥ 4 words` and lexical ratio ≥ 0.7 . Short but essential tails — "before?", "project.", "queries daily?", "In simple terms." — fail that test and are thrown away.

9. RAG / Rack Analysis — the central question of this audit

The premise is confirmed by production data, not hypothetical. 21 real transcripts contain "Rack" where the interviewer said RAG:

"In my Rack pipeline, there is a multiple"
"So you work a lot with LLMs and Rack systems."
"Let's move to the Rack. So explain the Rack architecture from document injection to final answer."
"Okay, so why did you use Rack instead of fine-tune and LLM?"
"Your Rack chatbot is answering correctly for 80% of queries."
"Okay, so your Rack chatbot currently takes 6 to 8 seconds to respond."

The same corpus contains Docker → "doctor", FastAPI → "fast repair", HTTP → "H3B", React → "Recture" / "Reacty", hierarchical RAG chatbot → "hierarchical drag chart bot".

9.1 Which layer is at fault?

Test F settles this. On US-English speech, RAG survived every degradation tested — noise to 6 dB SNR, telephone band-limiting, 35 % level, clipping. **The RAG → Rack error is therefore not caused by channel quality; it is conditioned on speaker accent and prosody.** That matters for the fix: no amount of audio cleanup, gain staging or VAD tuning will remove it. It has to be repaired in text.

9.2 What the system actually does with it — live model, résumé and JD both naming RAG

Transcript	Layer	Answer produced	Verdict
"What is Rack?"	LLM	"Rack is a minimal Ruby interface that standardises how web servers communicate with Ruby web frameworks..."	FAIL — answered a different technology
...then "Why would you use it?"	Context	"Using Rack lets you swap the web server or middleware... benchmark Nginx versus Puma..."	FAIL — the error propagated
"What is red is?"	LLM	"RAG stands for Retrieval-Augmented Generation..." (5 bullets)	FAIL — answered a <i>third</i> unrelated topic
"What is fast API?"	LLM	"FastAPI is a modern Python web framework..."	RECOVERED
"So you work with a doctor. Do you remember right about doctors?"	LLM	"Yes, I collaborated with a physician to design a medical-focused RAG chatbot..."	FAIL + fabrication
"Okay, so give me an idea about these H3B methods."	LLM	"I haven't come across a specific 'H3B' method... so I assume it's a typo for a known technique like RAG..."	FAIL — invented a topic
"Okay, you work in a deployment pack?" (prod)	LLM	Docker images, ECR, GitHub Actions, EC2	RECOVERED
"...hierarchical drag chart bot?" (prod)	Context	Two-tier vector index, correct continuation of the prior turn	RECOVERED

9.3 The pattern

Recovery succeeds when the mis-transcription is **phonetic noise** — "fast API", "deployment pack", "drag chart bot" are not words the model can take seriously, so it reaches for the nearest real term.

Recovery fails when the mis-transcription is **itself a legitimate technical or English word** — *Rack* (a real Ruby framework), *doctor*, *red is*, *H3B*. The model has no reason to doubt a coherent reading, so it answers it confidently.

There is a third, quieter failure: **the wrong word leaks into the spoken answer even when the meaning is recovered**. Verbatim from production:

Q: "Okay, you build any Rack chatbot?"

A: "I built a Rack-style chatbot for a client using LangChain's FSM workflow..."

Q: "...you work with a fast repair. Could you give me more idea about fast repair?"

A: "FastAPI is a modern Python web framework... I use it for fast-repair style services..."

The candidate reads that aloud in the interview.

10. Follow-Up Question Analysis

All five scenarios from the brief, plus a four-turn chain, run against the live model with a real résumé.

#	Turn 2	Gate reason	Resolver ran?	Outcome
A	"Why would you use it?"	follow_up	yes	PASS — "it" resolved to RAG, answered why not what
B	"How did you solve that?"	follow_up	yes	PARTIAL — coherent with turn 1, but turn 1 itself was truncated to "in your project?" (§14.3)
C	"What database did you use?"	question_mark	no — history only	PASS — "I used PostgreSQL as the primary relational store"
D	"What about Django?"	question_mark	no — history only	UNTESTED — the call hit a provider rate limit (§22)
E	"Can you give me an example?"	question_mark	no	FAIL — returned a 5-bullet dump of the candidate's projects, not an example of an API
K1	"Why would you use it?"	follow_up	yes	PASS
K2	"What are the disadvantages?"	question_mark	no — history only	PASS — correctly about RAG's limits
K3	"Have you used it?"	follow_up	yes	PASS — grounded in the résumé project
T	"Why." (period, not question mark)	too_short	—	FAIL — held 4 s, then silently discarded. No answer at all.

10.1 Two different follow-up paths, with different quality

Only `reason == "follow_up"` triggers `_resolve_followup()`, which rewrites the question into *"this is about what you JUST said... the previous question was X and you answered Y... don't repeat the same answer."* Everything else relies on the last three Q&A turns sitting in the user message. In practice the weaker path *usually* works (C, K2 passed) but it is what failed in E.

10.2 Follow-up detection is punctuation-sensitive VERIFIED

`_is_followup_question` strips a trailing "?" before matching its phrase set, but not "." or "!". Whisper emits "." on falling intonation, which is how a flatly-spoken follow-up arrives.

Phrase	with "?"	with "."	Impact of the "." form
Why	follow_up → answered	too_short → discarded	No answer at all
How	follow_up	too_short	No answer at all
Go on	follow_up	too_short	No answer at all
For example	follow_up	too_short	No answer at all
And why	follow_up	too_short	No answer at all
Tell me more	follow_up	question_prefix	Answered, but as a fresh question — no follow-up rewrite
Explain that	follow_up	question_prefix	Answered without the "about what you just said" instruction
What about it	follow_up	question_prefix	Same
Can you elaborate	follow_up	question_prefix	Same

Measured frequency: 67.5 % of the 1,441 real transcripts end in "."; 23.0 % end in "?". Of 818 short utterances (≤ 4 words), 668 ended in "." and only 77 in "?". Scanning the corpus for the specific phrases affected found **2 verbatim instances** ("Please elaborate.", "Give me the code for that.") — so the mechanism is certain but its observed rate in this corpus is low. Priority is set accordingly (P1, not P0).

10.3 Phrases missing from the follow-up set

"go on" is present but only matches without punctuation; genuinely absent are: "anything else", "okay and then", "what else", "can you give me an example" (the set has "can you give an example" — no "me"), "and after that", "then what".

11. Context Recovery Analysis

Does later context repair an earlier mis-transcription? Three controlled pairs, live model:

Turn 1 (misheard)	Turn 2	What the follow-up was actually about	Recovered?
"What is Rack?" (meant RAG)	"Why would you use it?"	Swapping Ruby web servers, Nginx vs Puma	NO
"What is red is?" (meant Redis)	"Why did you use it?"	RAG chunking, Chroma, hallucination rates	NO
"What is fast API?" (meant FastAPI)	"Why did you choose it?"	FastAPI async, OpenAPI, Pydantic validation	YES

The critical structural finding: context flows *forward* only. Once turn 1 is answered wrongly, that wrong answer is written to history by `add_history()` and becomes the ground truth the follow-up is resolved against. `_resolve_followup()` literally instructs the model: *"The previous question was 'What is Rack?' and you answered: 'Rack is a minimal Ruby interface...'. Directly justify, expand on, or clarify that specific answer."* **The follow-up mechanism actively locks in the error.** Nothing in the system can revise a past turn.

12. Self-Correction Analysis

This is the weakest area in the whole system. Three of three corrections failed.

12.1 "Tell me about your Flask project... actually, I mean my Django project." VERIFIED

```
[ 0.00] SPOKEN 'Tell me about your Flask project.'
[ 2.21] heard 'Tell me about your Flask project.'
[ 2.24] GATE interview_intent ANSWER ← fires immediately: complete + "looks finished"
[ 2.36] SPOKEN 'Actually, I mean my Django project.'
[ 4.88] heard 'Actually, I mean my Django project.'
[ 9.04] GATE fast_intent_wait hold ← held, then dropped
[10.13] ANSWER "I built a Flask-based RAG chatbot that pulls product documentation..."
                                         ← DJANGO WAS NEVER ANSWERED
```

Swept across pause lengths from 0.2 s to 4.0 s: **the outcome is identical at every one**. The first clause always wins because it is a grammatically complete request; the correction always arrives after the answer has already started.

12.2 "Which database did you use? MySQL. Sorry. I mean PostgreSQL." VERIFIED

Through real audio the VAD produced **four** separate utterances. Two distinct failures, depending on whether the fragments land in the same buffer:

- **If they merge:** the gate's focus selection picks the last strong clause ("Which database did you use?") and then refuses to grow into "MySQL Sorry." because it is only 2 words — so the LLM receives the question *without* the correction. Measured focus: 'Which database did you use?', dropped: 'MySQL Sorry. I mean PostgreSQL.'
- **If they don't merge:** "I mean PostgreSQL." is classified `keyword_match`, which **cancels the answer that is already streaming** and starts a new one about the bare word "PostgreSQL". Observed live: `answer_cancelled` at 5.90 s followed by a fresh `answer_start`.

12.3 "I said the HTTPS methods." — correcting the system, not themselves VERIFIED in production

```
12:14:52 heard "Okay, so give me an idea about these H3B methods. What is this?"
          ANSWER "I'm not familiar with a specific 'H3B' method... I'll assume it refers to a
                  hierarchical-based approach that combines three-stage heuristics for RAG..."
12:14:59 heard "I said the HTTPS methods."
          GATE fast_intent_ignore -> hold ← the correction was thrown away
12:15:09 heard "Okay, so give me an idea about HTTPS methods."
          ANSWER "GET retrieves a resource without side effects..." ← finally correct
```

Cost: three spoken turns and 17 seconds of interview time for one answer, plus one confidently wrong answer displayed in between. The pattern "I said ..." / "No, I meant ..." / "Why are not giving me an answer for the FAST API?" appears repeatedly in the real corpus — the interviewer is visibly working around the system.

12.4 Where the correction signal is lost

The words that mark a correction — **actually, sorry, I mean, no wait, I said, not X but Y** — are handled in exactly one place in the codebase: "actually " appears in `_FILLER_LEADS`, i.e. it is **stripped as noise** before the gate looks at the sentence. There is no rule anywhere that treats a correction as a replacement of a prior question, and no mechanism to revise a turn that has already been answered.

13. Pause Analysis

Test E, 5 two-part questions × 11 pause lengths, deterministic. **1** = merged correctly into one answer, **2** = split into two answers, **½** = only the first half answered and the real question discarded.

Question, split by a pause	0.2 s	0.6 s	1.0 s	1.6 s	2.0 s	3.0 s	4.0 s
"Can you explain..." / "RAG?"	2	2	2	2	2	2	2

Question, split by a pause	0.2 s	0.6 s	1.0 s	1.6 s	2.0 s	3.0 s	4.0 s
"What challenges did you face?" / "In your project?"	2	2	2	2	2	2	2
"What is the difference between..." / "FastAPI and Django."	1	1	1	1	2	2	2
"Will you give me the..." / "idea about your recent projects."	1	1	1	1	½	½	½
"Tell me about your Flask project." / "Actually, I mean my Django project."	½	½	½	½	½	½	½

13.1 Why the first two split at every pause length

`_should_keep_waiting()` has two unconditional release paths that run before any timing is considered:

```
if gate.reason == "follow_up":
    return False
if gate.complete and _looks_finished(...):
    return False
```

← "Can you explain?" is IN the follow-up phrase set
← "What challenges did you face?" ends in "?"

Both are correct for their intended case (a real bare follow-up must not be delayed; a barge-in must not be delayed) and both are wrong when the phrase happens to be the *opening* of a longer sentence. **No pause length can save these.**

13.2 Why the next two split at ~1.8 s

An answerable-but-unfinished fragment gets `question_soft_wait_seconds = 1.5 s`, measured from the last fragment, plus one 180 ms debounce cycle. If the speaker resumes before that expires, `speech_active` holds the buffer and everything merges. If they think for two seconds, the fragment is answered alone — and in the "Will you give me the..." case the continuation then arrives to a cleared buffer and is dropped as `no_question_signal` before it can be answered.

Threshold measured between 1.6 s and 2.0 s.

This is confirmed in production. From `default_2026-08-23.jsonl` at 06:43:53:

```
heard "Will you give me the..." → GATE question_prefix → ANSWER at slf=1673 ms
heard "idea about your recent projects." → GATE fast_intent_answer → ANSWER (a second time)
```

Two LLM calls, two answers streaming into the same overlay, for one spoken question.

13.3 Real audio confirms the VAD split is unavoidable at this setting

Through live Whisper, a 600 ms pause inside "Can you explain ... RAG?" produced two utterances and the transcripts "Can you explain?" + "RAG." — note that Whisper **added a question mark to the first half**, which is what makes the gate treat it as a finished question.

14. Question Gate Analysis

14.1 Decision distribution over 1,019 real gate decisions

Reason	Count	Answers?	Assessment
question_mark	1,103	yes	The workhorse. Also the main false-accept path (\$14.4).
question_prefix	239	yes	Sound, but fires on trailing-off fragments (\$14.2).
fast_intent_ignore	235	no	201 of these are from one day before the tier-2 token-budget fix; post-fix it is 6–23/day.
no_question_signal	228	no	Held for the 4 s continuation window then dropped.
too_short	157	no	Mostly correct (noise); also swallows "Why." (\$10.2).
interview_intent	142	yes	Good recall on opener requests; truncates specific questions (\$14.3).
follow_up	104	yes	Works well, but never waits (\$13.1).
small_talk	67	no	Correct.
keyword_match	55	yes	Explicit product trade-off; it also cancels running answers (\$14.5).
incomplete_fragment	27	no	Only reachable when nothing else matched first.
fast_intent_answer / wait	4 / 4	yes / no	Tier-2 rarely changes an outcome even after the fix.
question_phrase	1	yes	Effectively dead — the phrase list almost never wins.

14.2 Trailing-off fragments are classified as answerable VERIFIED

`_classify_clause` sets `complete = not _trails_off(...)` but **still returns** `should_answer=True` for anything matching a question prefix. The `complete` flag only buys 1.5 s of grace. Measured with the real wait policy:

fragment	complete	0.2s	1.0s	1.6s	4.5s	result at 1.6s
'Can you explain...'	False	HOLD	HOLD	FIRE	FIRE	LLM called with "Can you explain..."
'What is the difference between...'	False	HOLD	HOLD	FIRE	FIRE	LLM called with the fragment
'How did you...'	False	HOLD	HOLD	FIRE	FIRE	LLM called with the fragment
'Explain the'	False	HOLD	HOLD	FIRE	FIRE	LLM called with "Explain the"
'Tell me about your'	False	HOLD	HOLD	FIRE	FIRE	LLM called with "Tell me about your"
'What is RAG?'	True	FIRE	FIRE	FIRE	FIRE	correct – fires at once

14.3 The interview-intent matcher hijacks and truncates specific questions VERIFIED

`_match_interview_intent` scans every word window for a fuzzy match (ratio ≥ 0.82) against phrases like "tell me about your project". When it matches mid-sentence, `_focus_from_span` **discards everything before the match**. Because the surviving text has a high lexical ratio, `_resolve_interview_intent()` is skipped too — so the LLM receives the bare fragment with no explanation.

What the interviewer asked	What the LLM received	Lost
What was the biggest challenge in your project?	"in your project?"	"What was the biggest challenge"
What tech stack did you use in your project?	"in your project?"	"What tech stack did you use"
What testing did you do in your project?	"in your project?"	"What testing did you do"
How many people were in your project?	"in your project?"	"How many people were"
What was the hardest bug in your project?	"in your project?"	"What was the hardest bug"
How long was your experience with Django?	"was your experience with Django?"	"How long"
Can you describe your experience with Kubernetes?	"describe your experience with Kubernetes?"	"Can you" (harmless)

6 of 15 realistic project/experience questions lost their interrogative. Confirmed end-to-end: in the live run, "What was the biggest challenge in your project?" produced a five-bullet generic project summary that never mentioned a challenge.

14.4 False accepts VERIFIED, from real transcripts

Real transcript	Gate	What happened / would happen
"Okay, would I?"	question_mark	Answered in production: <i>"I'm confident my experience... so I'd be a strong fit."</i> Nobody asked anything.
"I don't know what I'm doing."	question_prefix	Focus becomes <i>"what I'm doing."</i> and is answered
"So it's Docker."	keyword_match	Answered as <i>"tell me about Docker"</i>
"Corrections, Frappe."	keyword_match	Answered as <i>"tell me about Frappe"</i>
"But you haven't mentioned Docker in your resume."	no_question_signal	Correctly held — the 4-word content-clause rule saved it

`_question_mark_is_credible` passes any clause of ≥ 3 words. "Okay, would I?" is three words after filler stripping, so it qualifies.

14.5 keyword_match cancels a running answer VERIFIED

A bare term mention is `should_answer=True`, and in `_process_after_debounce` any confirmed non-follow-up answerable text cancels the in-flight answer. Observed live: the answer to "Which database did you use?" was cancelled 3.9 s in by the fragment "I mean PostgreSQL."

14.6 Gate CPU cost

0.57 ms per case average over 132 utterances — negligible, and the per-buffer memoisation is doing its job. Not a latency concern.

15. LLM Understanding Analysis

The system prompt carries the entire burden of transcript repair, in one sentence:

"The text comes from live speech-to-text and may contain small errors. Silently infer the most plausible real term from the resume/JD context and answer that; never mention the transcription or ask them to repeat."

Measured behaviour of that instruction across this audit:

Situation	Rate	Notes
Mis-transcription is obvious nonsense	recovers	"fast API", "deployment pack", "drag chart bot", "H3B"→(tried), "fast repair"
Mis-transcription is a real word/technology	fails	"Rack"→Ruby Rack, "doctor"→physician, "red is"→answered RAG instead
Recovers meaning but repeats the wrong term aloud	observed 3×	"Rack-style chatbot", "fast-repair style services", "doctor-provided HL7 feeds"
Says it doesn't recognise the term, then invents one	observed 2×	"H3B" → "hierarchical three-stage heuristics"
Asks the interviewer to repeat	never	Explicitly forbidden by the prompt. There is no "I may have misheard" path at all.

The résumé and JD are in the prompt but are used as *background*, not as a **vocabulary constraint**. In the failing "Rack" run, both documents named RAG explicitly and the model still chose Ruby Rack.

16. Answer Quality Analysis

16.1 Fabrication — the most serious answer-quality defect VERIFIED

Across 33 answers generated with a fixed, known résumé and JD:

- **28 of 28 quantified claims** appear nowhere in the supplied documents.
- **13 of 17 named tools/companies** appear nowhere in them.

Examples, all first-person claims the candidate would have to defend:

```
"cut support ticket handling time by about 30 %"      résumé says only "reduced ... time", no number
"lowered query latency from roughly 800 ms to under 200 ms"
"the Docker image stayed under 120 MB"
"cut ticket resolution from fifteen minutes to under five"  ← contradicts the 30% claim above
"the documentation set grew past 200 GB"
"hallucination rates ... roughly 12 % without it versus under 3 % with it"
"reduced code-review cycles by about 30 percent"
"a GitHub Actions job that runs a synthetic load test with Locust after each build,
  aborting deployments if 95th-percentile latency exceeds 250 ms"
```

Production shows the same behaviour with a different résumé: "I've integrated clinical data APIs for a health-tech client... translating doctor-provided HL7 feeds", and RAG answers naming Pinecone, Azure Cognitive Search and Mistral embeddings.

The prompt's own words: "Only name a specific company, project, tool, metric or achievement if it actually appears in the resume or job description — that is the one hard line." Two other prompt rules pull directly against it: "Every bullet earns its place with at least one of: a named component, tool or method; a number, threshold or scale..." and "Name the actual thing, never its category." When the résumé has no number to hand, the model satisfies the depth rule by inventing one.

16.2 Format rules the model does not follow VERIFIED

Prompt rule	Compliance	Measured
"One simple thing asked: 1–3 plain spoken sentences, no bullets "	6 / 18	12 of 18 single-part questions came back as bullet lists
"3–5 bullets (max 6)"	33 / 33	Never exceeded
"Each bullet is ONE complete spoken sentence of 15–35 words"	mostly	Sentence length p50 = 27, p90 = 34, max = 41
"40 words for a single-part question, 90 across bullets"	15 / 33	18 answers (55 %) exceeded it; p50 = 107 words, max = 173
Banned words (leverage, robust, furthermore, moreover...)	32 / 33	One "leverage" in 33 answers — the vocabulary rules work well
"Code goes in a fenced block"	2 answers	Inline backticks used instead — unreadable when spoken

16.3 Characters that do not survive being read or pasted VERIFIED

88 non-ASCII dashes and narrow no-break spaces and **25 smart quotes** across 33 answers — Retrieval-Augmented (U+2011), 30 % (U+202F), 've. On screen they look fine; pasted into a chat box or a code comment they frequently do not, and they make the text visually harder to scan quickly under interview pressure.

17. Human Naturalness Analysis

Scored 0–10 for: could a real candidate say this out loud, in this order, without sounding like they were reading?

Question	Answer (opening)	Score	Why
"What is RAG?"	"RAG stands for Retrieval-Augmented Generation, where a language model first pulls relevant documents from a vector store like Chroma and then generates answers using that retrieved context..."	9	34 words, one sentence, plain, speakable. This is the target.
"What is an API?"	"An API is a set of defined endpoints that let one software component call functions or retrieve data from another..."	9	30 words, direct, no preamble.
"What database did you use?"	"- I used PostgreSQL as the primary relational store for all our backend services..."	7	Right content, but a one-item bullet list for a one-word answer reads oddly.
"Why would you use it?"	5 bullets, 140 words, each 25–30 words	5	~55 seconds spoken. An interviewer stops listening. Uniform bullet lengths read as generated.
"Have you used it?"	5 bullets, 154 words, with "cosine similarity thresholds above 0.78", "200 GB", "15 percent"	3	Longest answer to the shortest question; unverifiable invented numbers the candidate cannot defend.
"Can you give me an example?"	5 bullets of the candidate's whole project history	3	Does not answer what was asked.
Scenario: disk full	4 bullets with <code>df -h</code> , <code>docker image prune -a</code> , <code>resize2fs</code>	8	Genuinely good technically; inline backticks are awkward to read aloud.
"So you work with a doctor..."	"Yes, I collaborated with a physician to design a medical-focused RAG chatbot..."	1	Fluent, natural — and completely false.

Aggregate naturalness: 5.5 / 10. Short definitional answers are excellent — genuinely close to the style the brief asks for. The failure is length discipline on follow-ups and the reflex to reach for bullets. A 107-word median answer is roughly 45 seconds of speech; the prompt's own 40/90-word budget is the right target and is being exceeded in 55 % of answers.

18. Indian English / Accent Analysis

Method note, stated plainly: this machine has only two US-English SAPI voices (David, Zira). **No Indian-accent speech could be synthesised**, so the synthetic tests measure the *easy* case and are an upper bound. The accent evidence in this section therefore comes entirely from the 1,441 real transcripts, which *are* Indian-English speech on real calls — a better source than any synthetic proxy.

Measure over the real corpus	Value	Reading
Transcripts with lexical word ratio < 0.75	154 (11 %)	One in nine utterances came back garbled enough to trip the "terse or garbled" rule
Transcripts flagged low confidence (< 0.55)	76 (5 %)	
Transcripts containing non-Latin script	17 (1.2 %)	Whisper switched to Devanagari mid-call
Technical terms mis-transcribed	RAG→Rack (21×), Docker→doctor, FastAPI→fast repair, HTTP→H3B, React→Recture/Reacy, Frappe→"Corrections, Frappe"	

A verbatim Hinglish failure from `default_2026-08-23.jsonl` at 07:59:06:

```
heard "यूबेर फिदा Fashtrape है, को लिदो के विदे आईडिया वर्ट फास्ट रेप है," conf=0.41
GATE fast_intent_ignore -> hold <- ignored entirely
(40 seconds later, the interviewer rephrases in English and finally gets an answer)
```

With `STT_LANGUAGE=en` set, Whisper still switched scripts. The transcript survived the quality filter — the non-Latin branch of `lexical_word_ratio` deliberately treats such tokens as real words — but the gate has no rule that can read it, tier-2 said IGNORE, and the utterance was lost.

Other real-corpus patterns worth naming: the interviewer's name is routinely re-invented by the recogniser ("Vishal", "Vishay", "Arisha", "Ujjaj", "Fisad", "Nuj" for "Uday"), and Indian-English question forms without a leading interrogative ("Docker you have used or not?", "Tell na, what is Redis?") are handled correctly *only* when Whisper supplies a "?".

19. Latency Analysis

19.1 Real sessions

Segment of the budget	p50	p90	p99	Source
VAD end-of-speech silence (fixed cost)	420 ms	420 ms	420 ms	<code>segment_end_silence_ms</code>
STT round trip	254 ms	307 ms	467 ms	latest real session, n=53
Last word → LLM call starts	160 ms	195 ms	1,067 ms	<code>since_last_fragment_ms</code> , n=42
LLM call → first visible word	598 ms	883 ms	1,622 ms	n=42
Perceived: silence → first word	≈ 1.2 s	≈ 1.5 s	≈ 3.1 s	sum of the above
Full answer streamed	924 ms	1,172 ms	3,221 ms	n=42

This is good, and the 30 ms settled-debounce optimisation is visibly working: `since_last_fragment_ms` drops from ~185 ms to 31–45 ms on turns where the VAD had already settled.

19.2 Latency by question type **OBSERVED**

Gate reason	n	wait p50	first token p50	Total p50
follow_up	2	39 ms	506 ms	545 ms
question_mark (first question)	16	182 ms	815 ms	996 ms
question_prefix	8	182 ms	811 ms	994 ms

Gate reason	n	wait p50	first token p50	Total p50
interview_intent	2	116 ms	1,052 ms	1,169 ms
keyword_match	3	188 ms	1,941 ms	2,129 ms

Follow-ups are the fastest path, not the slowest. The concern in the brief that follow-ups add latency is not borne out — the extra history in the prompt costs nothing measurable, and the follow-up branch skips the soft wait entirely. The slow paths are the *ambiguous* ones: tier-2 classification adds up to 1.2 s, and the soft wait adds up to 1.7 s.

19.3 The real latency risk is provider throttling VERIFIED

Measured prompt size for one question: **1,396 tokens with no history, 1,684 tokens with three turns** (system prompt alone is 4,744 characters). Groq's reported limit for `openai/gpt-oss-120b` on this account is **8,000 tokens per minute**, and output tokens count too — so roughly **4–5 questions per minute** before throttling.

This audit triggered 429s repeatedly with only 2–4 questions per conversation. When it happens, the answer does not fail visibly — it **stalls**. Measured during testing: first-token latency of 6.1 s, 7.8 s, 9.7 s and 15.5 s against a 0.5 s baseline. The production logs show the same signature: `pipeline_ms` p99 of 20.7 s and a maximum of 125.8 s.

```
groq.RateLimitError: 429 - Rate limit reached for model `openai/gpt-oss-120b` ...  
on tokens per minute (TPM): Limit 8000, Used 7195, Requested 1585.
```

20. Miscommunication Test Matrix

Observed behaviour only. Blank cells mean the case was not reachable in this audit and is not guessed at.

Intended	Transcript	Context	Gate	System interpretation	Correct?	Source
RAG	Rack	résumé + JD both name RAG	question_mark	Ruby Rack web-server interface; follow-up continued into Nginx/Puma	NO	Test D
RAG	Rack Pipeline	RAG résumé	question_prefix	Full, correct RAG pipeline (chunking → embeddings → retriever → deploy)	YES	prod 08-23
RAG	Rack chatbot	RAG résumé	question_mark	Correct content, but answered " <i>I built a Rack-style chatbot</i> " aloud	PARTIAL	prod 08-23
RAG	Rite chatbot	none	question_mark	Reached the gate as a valid scenario question	—	Test C (fast speech)
RAG	drag chart bot	prior turn about hierarchical RAG	question_mark	Correct — continued the two-tier index discussion	YES	prod 08-23
Redis	red is	Redis in résumé	question_mark	Answered about RAG — a third, unrelated topic	NO	Test D
Redis	Reedy's	—	question_mark	Reached the gate intact as a question	—	Test F (4/10 conditions)
API	happy	—	question_mark	Gate accepts; LLM not exercised	—	Test A
API	A.P.I.	—	question_prefix	Question truncated at the acronym — "design you used" dropped	NO	Test A2
AWS	A.W.S.	—	question_prefix	"before?" dropped	NO	Test A2
AWS	A B S	—	question_prefix	Accepted as a question; term not repaired anywhere	—	Test A
SQL	S.Q.L.	—	question_prefix	"queries daily?" dropped	NO	Test A2
SQL	sequel	—	question_mark	Accepted; no normalisation layer	—	Test A
Docker	doctor	Docker in résumé	question_mark	"I collaborated with a physician... HL7 feeds" — wrong AND fabricated	NO	Test D + prod
Django	Gango / D'Ango	Django in résumé	interview_intent	Reached the gate as an intent match	—	Test F (4/10)
Django	jango	—	interview_intent	Accepted	—	Test A
FastAPI	fast API	FastAPI in résumé	question_mark	Correctly recovered as FastAPI, follow-up correct too	YES	Test D
FastAPI	fast repair	FastAPI résumé	question_mark	Recovered, but said " <i>fast-repair style services</i> " aloud	PARTIAL	prod 08-23
Flask	Vlesk/Blask/Black	—	question_mark	Not in the STT vocabulary; broke in 8/10 channel conditions	NO	Test F
HTTP	H3B	—	question_mark	"hierarchical three-stage heuristics for RAG" — invented	NO	Test D + prod
Kubernetes	Kubernetes	—	question_mark	Reached the gate intact; LLM would almost certainly recover	likely	Test F (9/10)
LLM	L.M. / element	—	question_prefix	Accepted as a question; not repaired	—	Test A
GitHub	get hub	—	question_prefix	Accepted; not repaired	—	Test A

Intended	Transcript	Context	Gate	System interpretation	Correct?	Source
"why did you choose"	"why didn't you choose"	—	question_mark	Polarity inverted; nothing detects it	NO	Test F (3/10)

21. Failure Root Causes

Each failure traced through the full chain and attributed to the layer that could have prevented it.

RC-1 "Can you explain ... RAG?" → two answers

```
meant: one question          audio: 1.3 s + 0.6 s pause + 0.6 s
VAD:  2 segments, utterance_id 1 and 2, BOTH is_final=True  ← grouping does not apply to pauses
STT:  "Can you explain?" (conf 0.54) + "RAG." (conf 0.46) ← Whisper ADDED the "?"
filter:kept both
gate:  clause 1 matches the follow-up phrase set → reason=follow_up
wait:  `if gate.reason == "follow_up": return False` → fires at 33 ms, unconditionally
LLM:  answers "Can you explain?" from the résumé; then answers "RAG." separately
```

Layer: QUESTION_GATE (wait policy). The VAD gave no continuation signal and the wait policy had a hard-coded no-wait rule for the phrase class this fragment fell into.

RC-2 "Why did you choose ... FastAPI?" → topicless answer

```
VAD:  2 segments
STT:  "Why did you choose?" + "FAST API." (confidence below 0.35)
filter:DROPPED "FAST API." as low_confidence ← the topic is deleted here
gate:  "Why did you choose?" ends in "?" → complete → _looks_finished → fire immediately
LLM:  "I chose that stack because FastAPI lets me spin up lightweight async endpoints..."
```

Layer: TRANSCRIPT_FILTER, with QUESTION_GATE as the amplifier. A short but correct fragment was deleted because short fragments intrinsically score low confidence, and the gate then treated the remaining half as a finished question.

RC-3 "Tell me about your Flask project. Actually, I mean my Django project." → answers Flask

```
gate:  clause 1 = interview_intent, complete, ends in "." → _looks_finished → fire at 2.24 s
       clause 2 arrives at 4.88 s to an empty buffer; on its own it is a statement
tier-2:WAIT → held → dropped
```

Layer: QUESTION_GATE. No correction vocabulary exists ("actually" is stripped as a filler lead), and there is no mechanism to revise an answered turn.

RC-4 "What is Rack?" → Ruby Rack

```
audio: fine    VAD: fine    STT: "Rack" (a real word, high confidence)
filter: fine   gate: question_mark, correct
context: résumé and JD both name RAG and were both in the prompt
LLM:  chose the literal reading; history then locked it in for the follow-up
```

Layer: LLM_UNDERSTANDING, enabled by the absence of any TRANSCRIPT_REPAIR layer. Every upstream stage behaved correctly.

RC-5 "What was the biggest challenge in your project?" → generic project summary

```
STT:  correct    gate: _match_interview_intent matched the window "in your project" (ratio ≥ 0.82)
focus: _focus_from_span discarded every word before the match → "in your project?"
rewrite: skipped, because lexical_word_ratio("in your project?") = 1.0 ≥ 0.75
LLM:  received the literal string "in your project?"
```

Layer: QUESTION_GATE (focus selection).

RC-6 Five-sentence scenario → two answers, setup lost

```
3.92 s  buffer = "Okay, so let's take a scenario. Your production server's disk partition
          is 100% full." → gate fired on the SETUP and cleared the buffer
10.31 s  buffer = "The application has stopped responding. And the monitoring dashboard shows
          no alerts. How would you troubleshoot this?"
          → answered without the disk-full fact, which had already been consumed
```

Layer: QUESTION_GATE. Same root cause as RC-1/RC-3: a grammatically complete sentence releases the buffer regardless of whether a question has been asked yet.

RC-7 Invented metrics

```
context: résumé states outcomes without numbers
prompt:  "the one hard line" forbids inventing them
prompt:  ALSO requires "a number, threshold or scale" in every bullet
model:   satisfied the depth rule by inventing the number
```

Layer: ANSWER_GENERATION. Two prompt rules in direct conflict, with no post-generation check.

Failure attribution across everything found

Layer	Failures	Notes
AUDIO_CAPTURE	0	Not exercised end-to-end here; risks are known and instrumented
VAD	1	Utterance grouping does not cover natural pauses — an <i>enabler</i> of the gate failures more than a failure itself
STT	4	Missing vocabulary (Flask), accent-conditioned term errors, polarity inversion, low confidence on short fragments
TRANSCRIPT_FILTER	1	Deletes a short correct fragment, taking the topic with it
QUESTION_GATE	9	The dominant source. Premature firing, focus truncation, punctuation sensitivity, clause shredding, false accepts, answer cancellation
CONTEXT	1	History locks in a wrong answer; nothing can revise a past turn
LLM_UNDERSTANDING	3	Literal reading of real-word mis-transcriptions; wrong term echoed into the answer
ANSWER_GENERATION	3	Fabrication, format-rule violations, length
NO FAILURE — recovered	8	fast API, deployment pack, drag chart bot, 4-turn RAG chain, PostgreSQL from history, RAG under all 10 degradations, silence-hallucination filtering, scenario setup when it merges

22. Findings — P0

F-01 A mid-question pause splits one question into two answers, or discards the real question P0

VERIFIED

Component	question_gate — <code>_should_keep_waiting</code> , with <code>vad.py</code> as enabler
Scenario	"Can you explain... [pause] ...RAG?" · "What challenges did you face? [pause] In your project?" · "Will you give me the... [pause] ...idea about your recent projects."
Expected	One merged question, one answer.
Actual	Patterns 1–2: two answers at every pause from 0.2 s to 4.0 s (11 lengths tested). Patterns 3–4: merge below 1.6 s, split or lose the real question above 2.0 s.
Root cause	(a) <code>if gate.reason == "follow_up": return False</code> fires with zero grace when a follow-up phrase opens a longer sentence. (b) <code>gate.complete</code> and <code>_looks_finished()</code> fires on any clause ending in "?" — and Whisper adds "?" to a rising-intonation fragment. (c) The VAD's <code>utterance_id</code> continuation signal only exists for > 8 s of unbroken speech, so it never covers a natural pause.
Evidence	Test E, 55 trials. Production <code>default_2026-08-23.jsonl</code> 06:43:53 — "Will you give me the..." answered at <code>slf=1673 ms</code> , then "idea about your recent projects." answered again. Live run L: two <code>answer_start</code> events 1.2 s apart.
Confidence	Certain — reproduced deterministically and present in production logs.
User impact	Two answers stream into the overlay for one question; the first is topicless. The candidate has to read and discard one live, mid-interview. Highest-frequency defect found.
Latency impact	Doubles LLM cost per affected question and consumes the TPM budget twice (see F-13).
Accuracy impact	Severe — the answer that fires first is frequently answering half a question.
Recommended	Add a short continuation window that applies to <i>any</i> answerable buffer, not only to incomplete ones: after firing eligibility is reached, wait ~250–400 ms for a same-turn continuation before releasing, and merge-and-supersede if one arrives (cancel the in-flight answer and re-ask with the merged text). Separately, do not treat a follow-up phrase as a follow-up when it is the <i>first</i> fragment of a buffer with no prior history.
Regression risk	High. Directly trades against the 182 ms p50 that was deliberately engineered. Barge-in responsiveness is bounded by the same code path.
Tests required	Re-run Test E (all 55 trials) · re-run the latency percentiles over ≥ 50 real answers · the existing <code>test_question_gate_scenarios.py</code> barge-in and <code>midquestion_pause</code> scenarios · a new scenario for "follow-up phrase with no history".

F-02 Self-corrections are never applied P0 VERIFIED

Component	question_gate — wait policy, clause focus, and the absence of correction vocabulary
Scenario	"Tell me about your Flask project. Actually, I mean my Django project." · "Which database did you use? MySQL. Sorry. I mean PostgreSQL." · "I said the HTTPS methods."
Expected	The final meaning wins: Django, PostgreSQL, HTTPS.
Actual	0 of 3 corrections applied. Flask answered and Django never answered, at every pause length 0.2–4.0 s. The HTTPS correction was gated <code>fast_intent_ignore</code> in production and thrown away; the interviewer had to ask a third time (17 s lost).
Root cause	The first clause is complete, so it fires before the correction is spoken. On its own, a correction is a statement, so tier-1 finds no question signal and tier-2 says WAIT/IGNORE. "actually" is in <code>_FILLER_LEADS</code> and is stripped as noise. No mechanism exists to revise an answered turn.
Evidence	Test D run O (full event trace) · Test E all 11 gaps · production 12:14:59.
Confidence	Certain.
User impact	The candidate is shown a confident, detailed answer to the question the interviewer explicitly retracted. This is the failure most likely to be visible to the interviewer.
Latency impact	Adds a full extra round trip when the interviewer repeats themselves; 17 s measured in production.
Accuracy impact	Total for the affected turn.
Recommended	Recognise correction markers (<i>actually, sorry, I mean, no wait, I said, not X but Y, let me rephrase</i>) as a distinct gate class that (a) suppresses immediate firing on the preceding clause for the continuation window from F-01, and (b) if an answer has already started, supersedes it — cancel and re-ask with the corrected text. This composes with F-01's merge-and-supersede rather than needing its own machinery.
Regression risk	Medium. "Actually, what is a race condition?" is a legitimate barge-in and must still cancel-and-answer, which is the same primitive — the risk is mis-classifying a genuine new question as a correction.
Tests required	New correction suite (≥ 8 forms $\times$ 5 pause lengths) · existing <code>barge_in</code> and <code>stray_word_during_answer</code> scenarios · verify "Actually, what is X?" still barges in.

F-03 A mis-transcribed technical term is answered literally, and the error propagates PO VERIFIED

Component	No component — this is a missing layer between STT and the LLM
Scenario	"What is Rack?" (RAG) · "What is red is?" (Redis) · "you work with a doctor" (Docker) · "H3B methods" (HTTP)
Expected	The intended term is recovered before the answer is generated, or the uncertainty is surfaced.
Actual	"Rack" → Ruby Rack middleware, and the follow-up "Why would you use it?" continued into Nginx vs Puma. "red is" → answered about RAG. "doctor" → invented a physician collaboration. "H3B" → invented "hierarchical three-stage heuristics". Recovery worked only where the wrong word was obvious nonsense (3 of 6).
Root cause	The only repair mechanism is one sentence in the system prompt. The résumé/JD are supplied as background, not as a vocabulary constraint. When the mis-transcription is itself a valid word, the model has no reason to doubt it.
Evidence	Test D runs F, G, I, J with a résumé and JD that both name RAG. 21 real "Rack" transcripts. Test F proves it is accent-conditioned, not channel-conditioned: RAG survived all 10 degradations on US-English speech.
Confidence	Certain.
User impact	A fluent, confident, completely wrong answer — the worst possible failure mode for an interview aid, because it is not obviously wrong on screen. Also leaks the mis-transcribed word into the spoken answer ("I built a Rack-style chatbot").
Latency impact	A normalisation pass costs ~0–5 ms if done lexically; a small LLM pass would cost 300–800 ms and should be avoided on the critical path.
Accuracy impact	The single largest accuracy defect in the system.
Recommended	A lexical term-normalisation layer between the filter and the gate: a phonetic/edit-distance map from observed mis-transcriptions to canonical terms (rack→RAG, red is→Redis, doctor→Docker, fast repair/fast API→FastAPI, get hub→GitHub, sequel→SQL, jango→Django, H3B→HTTP, kubernets→Kubernetes ...), restricted to terms that appear in the résumé, the JD, or the conversation history so it cannot fire on a genuine "Ruby Rack" question. Log every substitution. Additionally, pass the résumé/JD term list to the LLM explicitly as "the vocabulary this speaker uses", and forbid repeating a term back that is not in that list.
Regression risk	Medium-high. An over-eager map will corrupt correct transcripts — a real Ruby interview would break. Gating on résumé/JD/history presence is what keeps this safe.
Tests required	The 24-row matrix in §20 · a negative suite (Ruby/Rack, medical, retail vocabulary in the résumé — the map must NOT fire) · re-run Test D F/G/H/I · measure added gate CPU.

F-04 Answers invent metrics, tools and achievements that are not in the résumé P0 VERIFIED

Component	question_gate — _build_answer_messages system prompt; answer generation
Scenario	Any question inviting a first-person claim.
Expected	Only facts present in the résumé/JD, per the prompt's stated "one hard line".
Actual	28 of 28 quantified claims across 33 answers appear nowhere in the supplied documents; 13 of 17 named tools/orgs likewise. The same project was given "cut handling time by about 30 %" in one answer and "cut ticket resolution from fifteen minutes to under five" in another. Production shows an invented health-tech client and HL7 integration.
Root cause	Two prompt rules conflict: the anti-fabrication line versus "Every bullet earns its place with at least one of: a named component, tool or method; a number, threshold or scale...". With no number in the résumé, the model satisfies the depth rule by inventing one. Nothing verifies the output.
Evidence	Automated grounding check over 33 answers with a fixed known résumé/JD; verbatim production answers.
Confidence	Certain.
User impact	Career risk, not just quality risk. The candidate speaks a number they cannot defend; an interviewer who probes it exposes them. Inconsistent numbers across turns are self-incriminating.
Latency impact	None if fixed in the prompt; a verification pass would add latency and is not recommended on the critical path.
Accuracy impact	High.
Recommended	Rewrite the depth rule so a specific is required only <i>when one is available</i> , and state explicitly that a number not in the résumé must be phrased as a general engineering fact, never as something the candidate did. Consider a cheap post-generation check that flags digits absent from the résumé so the overlay can mark them.
Regression risk	Medium — the depth rules are what stop answers being vague, and the prompt notes they were tuned against a judged benchmark. Any change must be re-scored on that benchmark.
Tests required	Re-run the grounding check on ≥ 30 answers with a numberless résumé · re-run the prompt's own 8-shape × 7-rule benchmark before/after · verify answers do not become vague.

F-05 The interview-intent matcher truncates specific questions to a bare fragment P0 VERIFIED

Component	question_gate — <code>_match_interview_intent</code> + <code>_focus_from_span</code>
Scenario	Any question containing "in your project", "your experience", "your background".
Expected	The whole question reaches the LLM.
Actual	6 of 15 realistic questions were reduced to "in your project?". Confirmed live: "What was the biggest challenge in your project?" produced a five-bullet project summary that never mentioned a challenge.
Root cause	The matcher scans every word window; a mid-sentence match causes <code>_focus_from_span</code> to discard everything before it. <code>_resolve_interview_intent()</code> is then skipped because the surviving fragment has a high lexical ratio.
Evidence	15-case probe (6 truncations) · Test D run B end-to-end.
Confidence	Certain.
User impact	Generic canned-sounding answers to specific questions — a large share of "the answer felt generic" complaints likely trace here.
Latency impact	None.
Accuracy impact	High on a common question family.
Recommended	Only allow an intent match to <i>truncate</i> when the words before the match are not content (the same <code>_is_content_clause</code> guard <code>_locate_question_start</code> already applies). If the lead is content, keep the whole clause and treat the intent as advisory metadata only.
Regression risk	Low-medium. Must not lose the case this rule exists for: "Uday, tell me about yourself." with a junk prefix.
Tests required	The 15-case probe · the existing <code>garbled_interview_intent</code> scenario · verbatim production intents ("Ujjaj, let's start with your introduction...").

F-06 Multi-sentence scenario questions are answered twice, with the setup facts lost P0 VERIFIED

Component	question_gate — wait policy
Scenario	Setup sentences followed, after a pause, by the actual question.
Expected	One answer, reasoning from the stated facts.
Actual	Live run R: the setup alone was answered at 3.9 s; the real question was answered at 10.3 s without the disk-full fact , which had already been consumed. Production 08-23 08:02: "two users try to purchase the last available product..." answered, then "How will you prevent both orders from succeeding?" answered again.
Root cause	Same as F-01: a grammatically complete sentence releases the buffer whether or not a question has been asked. Tier-2's WAIT verdict exists precisely for this and is reached too rarely to help.
Evidence	Test D run R full trace; two production instances.
Confidence	Certain.
User impact	Scenario questions are the highest-value questions in a technical interview and this is where the system is least reliable.
Latency impact	Doubles cost; the wasted answer also consumes the TPM budget.
Accuracy impact	High — the final answer ignores the stated numbers and symptoms, which the prompt itself calls out as wrong.
Recommended	Fixing F-01's continuation window addresses most of this. Additionally: a declarative sentence that names no question should not fire on the <code>interview_intent</code> or <code>keyword_match</code> paths at all — require an interrogative signal for those to release the buffer.
Regression risk	Medium — could delay legitimate short answers.
Tests required	Test D run R · the existing <code>scenario_setup_then_question</code> harness scenario · production replay of the 5 scenario questions in the corpus.

23. Findings — P1

F-07 A short correct fragment is deleted as low-confidence, taking the topic with it P1 VERIFIED

Component	transcript_quality — min_confidence = STT_DROP_CONFIDENCE_THRESHOLD (0.35)
Scenario / Actual	"Why did you choose... [0.8 s] ...FastAPI?" — "FAST API." scored below 0.35 and was deleted. The question became "Why did you choose?" and was answered with a guess.
Root cause	Confidence is derived from avg_logprob, which is intrinsically noisy over few tokens. Measured: 0.45–0.55 for 0.3–0.9 s fragments vs 0.74 mean for ≥ 1.5 s.
Evidence	Test C pause_c; confidence-vs-duration table in §6.5.
Confidence	Certain (mechanism); frequency in production not separately measured.
Impact	Deletes exactly the word that carries the topic. Latency: none. Accuracy: high when it fires.
Recommended	Make the drop threshold duration-aware — do not hard-drop a fragment shorter than ~1 s on confidence alone when it arrives within the continuation window of an answerable buffer; mark it uncertain instead and let the LLM see it.
Regression risk	Medium — this threshold is part of what suppresses hallucinated text. Must be re-validated against the noise-only corpus.
Tests required	Noise-only corpus (hallucination rate must not rise) · Test C pause suite · re-measure transcript_dropped rate on the production corpus.

F-08 Follow-up detection is punctuation-sensitive P1 VERIFIED

Component	question_gate — _is_followup_question strips "?" but not "." or "!"
Actual	"Why." / "How." / "Go on." / "For example." / "And why." → too_short → held 4 s → silently discarded, no answer at all . "Tell me more." / "Explain that." / "What about it." → answered, but as a fresh question, so _resolve_followup() never runs.
Evidence	48-cell punctuation grid; Test D run T end-to-end (no answer produced).
Confidence	Certain (mechanism). Observed frequency in the real corpus is low: 2 instances in 1,441 transcripts — which is why this is P1 and not P0, despite 67.5 % of transcripts ending in ".".
Impact	Silent no-answer on a bare follow-up; weaker context binding on the others.
Recommended	Strip all terminal punctuation (. ? ! ...) before matching, and add the missing phrases: "anything else", "what else", "okay and then", "then what", "and after that", "can you give me an example".
Regression risk	Low — the set is exact-match and already narrow.
Tests required	The 48-cell grid · confirm "Why." with no history does not produce a contextless answer.

F-09 "Can you give me an example?" is not treated as a follow-up P1 VERIFIED

Component	question_gate — follow-up phrase set contains "can you give an example", not "...give me an example"
Actual	Scenario E: after "What is an API?", the follow-up produced a five-bullet dump of the candidate's projects instead of an example of an API.
Evidence	Test D run E, live model.
Confidence	Certain.
Impact	"Give me an example" is one of the most common interview follow-ups.
Recommended	Add the missing variants; better, match follow-ups by normalised shape (optional "me/us", optional "please") rather than by literal string.
Regression risk	Low.
Tests required	Scenario E end-to-end · the follow-up phrase grid.

F-10 Spelled-out acronyms and decimals shred the sentence and truncate the question P1 VERIFIED

Component	question_gate — _CLAUSE_BOUNDARY + _is_content_clause (≥ 4 words)
Actual	"Explain the A.P.I. design you used." → answers "Explain the A.P.I."; "Have you deployed on A.W.S. before?" → drops "before?"; "Do you write S.Q.L. queries daily?" → drops "queries daily?"; "The server runs at 99.9. percent uptime. How do you monitor it?" → drops the whole setup.
Evidence	11-case probe in §8.
Confidence	Certain (mechanism). Frequency depends on how often Whisper emits dotted acronyms — it did <i>not</i> in Test C (it normalised "R A G" to "RAG"), so real-world rate is probably low.
Impact	Silent truncation of the question with no signal anywhere.
Recommended	Do not split on a full stop preceded by a single letter or a digit. Separately, lower the neighbour-inclusion bar for a <i>trailing</i> clause that starts with a preposition/conjunction — "before?", "in your project?", "queries daily?" are continuations, not separate utterances.
Regression risk	Low-medium — clause splitting drives focus selection everywhere.
Tests required	The 11-case probe · the full Test A matrix (focus column) to confirm no focus changes elsewhere.

F-11 The résumé never reaches the recognizer when a JD is loaded; 39 common terms are missing from the vocabulary P1 VERIFIED

Component	audio_ws — _build_stt_prompt + Settings.stt_prompt
Actual	433 + 500 + 300 = 1,114 bytes truncated to 850; the résumé is the tail and is entirely discarded. Measured directly: résumé reached the recognizer: False .
Root cause	Fixed vocabulary first, JD second, résumé last, hard byte cap.
Evidence	Direct measurement (§6.3). Control case: Flask, absent from the vocabulary, broke in 8/10 degraded conditions while RAG and Docker held in 10/10.
Confidence	Certain.
Impact	The session's own project and tool names — the terms most likely to be spoken and most likely to be misheard — get no biasing at all.
Recommended	Build the prompt from extracted terms rather than raw prose: pull capitalised/technical tokens out of the résumé and JD, deduplicate against the static list, and fill the 850-byte budget with terms rather than sentences. Prioritise session-specific terms over the generic list.
Regression risk	Low — the prompt is advisory to Whisper.
Tests required	Re-run Test F with a résumé-derived vocabulary and confirm Flask survives · confirm no regression on RAG/Docker · byte-budget assertion test.

F-12 Answers exceed their own format and length budget P1 VERIFIED

Component	Answer prompt / answer generation
Actual	12 of 18 single-part questions answered with bullets (prompt says 1–3 plain sentences). 18 of 33 answers (55 %) over the prompt's 90-word ceiling; p50 = 107 words $\approx$ 45 s spoken; longest answer (154 words) was to the shortest question ("Have you used it?"). Sentence length p50 = 27 words. 88 non-ASCII dashes and 25 smart quotes across 33 answers.
Evidence	Automated style metrics over 33 live answers.
Confidence	Certain.
Impact	Directly against the brief's goal: answers should be speakable, short, natural. A 107-word bullet list is read, not spoken.
Recommended	Make the single-part/multi-part branch mechanical rather than judgemental (bullets only when the question contains more than one interrogative or is explicitly a scenario/comparison). Add an explicit hard ceiling in words for follow-ups. Normalise U+2011/U+202F/smart quotes to ASCII on the way out.
Regression risk	Medium — the prompt was tuned on a judged benchmark that must be re-run.
Tests required	Re-run the style metrics over $\geq$ 30 answers · re-run the prompt's own benchmark · human read-aloud check on 10 answers.

F-13 Provider throttling stalls answers by 6–15 s, and every duplicate answer makes it worse P1

VERIFIED

Component	Capacity — prompt size vs Groq TPM
Actual	1,396 tokens/question with no history, 1,684 with three turns, against an 8,000 TPM limit $\approx$ 4–5 questions/minute. Triggered repeatedly during this audit; first-token latency measured at 6.1 s, 7.8 s, 9.7 s, 15.5 s vs a 0.5 s baseline. Production <code>pipeline_ms</code> p99 = 20.7 s, max = 125.8 s.
Evidence	Live 429 payloads; measured message sizes; production percentiles.
Confidence	Certain.
Impact	The worst latency in the system by an order of magnitude, and it fails <i>silently</i> — the overlay just sits there.
Recommended	Three independent levers: (1) fix F-01/F-06 so the duplicate answers stop consuming budget; (2) surface throttling explicitly in the overlay instead of stalling; (3) the 4,744-character system prompt is the single largest per-question cost — the comment says it was already cut once against a benchmark, so any further cut must be benchmarked the same way.
Regression risk	Low for (1) and (2); high for (3).
Tests required	Sustained-rate test: 10 questions in 2 minutes, measure 429 rate before/after · confirm the overlay shows a throttle state.

F-14 A bare term mention cancels an answer that is still streaming P1 VERIFIED

Component	question_gate — keyword_match is should_answer=True, and any confirmed answerable text cancels the in-flight answer
Actual	Live: the answer to "Which database did you use?" was cancelled 3.9 s in by the fragment "I mean PostgreSQL." Real corpus contains many such fragments ("So it's Docker.", "Corrections, Frappe.", "Frappe, Frappe, framework.").
Evidence	Test D run P (answer_cancelled then a new answer_start).
Confidence	Certain.
Impact	A good answer is destroyed mid-sentence by an incidental mention. Real cancellation rate in production is 3 % of answers overall but reached 144 events in one day.
Recommended	Treat keyword_match as a weak signal that may start an answer when nothing is running but may never cancel one.
Regression risk	Low.
Tests required	Test D run P · the existing stray_word_during_answer and barge_in_new_question scenarios.

24. Findings — P2 / P3

ID	Pri	Component	Finding	Evidence	Recommendation
F-15	P2	question_gate	A spurious "?" on noise is accepted as a question. "Okay, would I?" was answered in production with an unsolicited "I'd be a strong fit" pitch.	VERIFIED prod 08-23 12:13:24; Test A	Require a verb or a known interrogative in _question_mark_is_credible, not just ≥ 3 words.
F-16	P2	question_gate	The mid-clause question-prefix scan fires inside statements: "I don't know what I'm doing." → answers "what I'm doing."	VERIFIED Test A	Require the lead-in before a mid-clause interrogative to end at a clause boundary or a comma.
F-17	P2	STT / gate	Whisper switches to Devanagari on Hinglish (17/1441 = 1.2 % of real transcripts); the gate cannot read it and tier-2 says IGNORE. One traced instance cost 40 s.	VERIFIED prod 07:59:06	Detect non-Latin output and either re-transcribe with an explicit language hint or route it to the tier-2 classifier as a translate-then-classify step.
F-18	P2	STT config	STT_FALLBACK_ENABLED=true with no OPENAI_API_KEY — there is no fallback. Any failure or 12 s timeout loses the segment.	VERIFIED warning on every audit run	Either set a key or set the flag false so the configuration stops implying redundancy that does not exist.
F-19	P2	VAD	Force-cut grouping glues unrelated continuous audio into one utterance — a song lyric was grouped with "But suddenly CPU reaches 90 to 95%."	VERIFIED prod 08-20 utt 16	Do not extend an utterance across a force-cut when the two halves share no lexical overlap; or cap grouping at 2 segments.
F-20	P2	Context	History is written only after an answer completes. A cancelled answer leaves the next follow-up with nothing to resolve against.	INFERRED from add_history placement; partially mitigated by the follow-up await path	Write a provisional history entry when an answer is cancelled after enough tokens to be meaningful.
F-21	P2	Answer gen	A reasoning model can spend the whole 500-token budget on hidden thinking and emit nothing. Handled with an explicit error, but the user simply loses that turn.	VERIFIED in code + the comment documenting measured empty answers	On an empty answer, retry once automatically with a higher budget rather than surfacing an error.

ID	Pri	Component	Finding	Evidence	Recommendation
F-22	P3	question_gate	Indirect asks are missed: "You know, I wanted to ask about RAG." → no_question_signal .	VERIFIED Test A	Add "I wanted to ask", "I was going to ask", "my question is" to the question-phrases list.
F-23	P3	question_gate	question_phrase fired once in 1,019 real decisions — effectively dead code that still costs a scan.	VERIFIED log analysis	Either extend the phrase list meaningfully or fold it into the prefix scan.
F-24	P3	Tier-2	The tier-2 classifier changed the outcome 8 times in 1,019 decisions post-fix (4 ANSWER, 4 WAIT) while costing up to 1.2 s on the critical path whenever it runs.	OBSERVED log analysis	Measure its value against its latency; it is the natural place to implement the F-02 correction logic, which would justify it.

25. Recommended Improvements — grouped by the problem they solve

#	Change	Fixes	Effort	Risk	Expected effect
1	Continuation window + merge-and-supersede in the wait policy, applied to every answerable buffer	F-01, F-02, F-06	Medium	High	The single highest-value change. Removes duplicate answers, half-questions and lost scenario setups. Costs ~250–400 ms on questions that are already unambiguous unless the release is made conditional.
2	Term-normalisation layer between the filter and the gate, gated on résumé/JD/history vocabulary	F-03	Medium	Med-high	Turns the largest accuracy defect from "silently wrong" into "corrected and logged". Near-zero latency if lexical.
3	Correction-marker recognition ("actually / sorry / I mean / I said")	F-02	Small	Medium	Composes with #1; without #1 it has nowhere to act.
4	Intent match may not truncate a content lead-in	F-05	Small	Low	Fixes a whole family of "generic answer" complaints for very little code.
5	Strip all terminal punctuation before follow-up matching; extend the phrase set	F-08, F-09, F-22	Small	Low	Cheapest real win in the report.
6	Do not split clauses on a dot preceded by a single letter or digit; keep short trailing continuations	F-10	Small	Low-med	Stops silent question truncation.
7	Duration-aware confidence drop	F-07	Small	Medium	Stops the topic word being deleted; must be re-validated against the hallucination corpus.
8	Build the STT prompt from extracted résumé/JD <i>terms</i> , not prose	F-11	Medium	Low	Attacks the mis-transcription problem at its source, complementing #2.
9	Resolve the fabrication/depth prompt conflict; normalise output punctuation	F-04, F-12	Medium	Medium	Removes career risk; makes answers speakable. Requires re-running the existing judged benchmark.
10	<code>keyword_match</code> may start but never cancel an answer	F-14	Tiny	Low	Stops good answers dying mid-sentence.
11	Surface throttling and low confidence in the overlay instead of stalling	F-13, F-03	Small	Low	Converts silent failure into a visible state the candidate can act on with the existing manual /ask override.

26. Regression Risks

Risk	Triggered by	Why it matters and how to hold the line
Latency regression	#1, #3, #7	The current 182 ms p50 was engineered deliberately (30 ms settled debounce, concurrent STT dispatch, cut system prompt). A continuation window is a direct tax on it. Hold: apply the window only when a continuation is plausible (buffer ends mid-phrase, or a correction marker was seen, or the last fragment was < 1 s of voiced audio) and release immediately otherwise; measure <code>since_last_fragment_ms</code> percentiles over ≥ 50 real answers before and after.
Barge-in becomes sluggish	#1	Barge-in shares the same release path. Hold: keep the immediate-release rule for a buffer that arrives while an answer is streaming.

Risk	Triggered by	Why it matters and how to hold the line
Over-eager term normalisation	#2	A map that fires unconditionally will destroy correct transcripts — a genuine Ruby, medical or retail interview breaks. Hold: only substitute when the canonical term is present in the résumé, JD or history, and log every substitution so it is auditable.
Hallucination rate rises	#7	The 0.35 drop threshold is one of three signals suppressing invented text. Hold: re-run the noise-only corpus and confirm the invented-transcript rate does not rise.
Answers become vague	#9	The depth rules are what stop generic answers, and the prompt notes they were tuned against a judged benchmark. Hold: re-run that benchmark; do not trim the prompt by eye.
Focus selection changes elsewhere	#4, #6	Clause splitting and focus growth drive every answered question. Hold: re-run the full 132-case matrix and diff the focus column, not just the pass/fail column.
More noise answered	#5	Extending the follow-up set widens what counts as answerable. Hold: re-run the 17 noise cases.

27. Testing Required Before Implementation

- Freeze the regression baselines that already exist.** Capture today's output of `client/test_question_gate_scenarios.py` (all scenarios, offline mode) and `client/test_pipeline_e2e.py` as golden files. Nothing in this report should be implemented before that exists.
- Adopt the audit harnesses as a permanent suite.** Test A (132-case matrix), Test A2 (mechanism probes), Test E (pause sweep, 55 trials) are deterministic, offline and run in seconds. They are the ones that will catch a regression in #1, #4, #5, #6.
- Build a labelled replay corpus from the production logs.** 1,441 real transcripts with their recorded gate decisions already exist. Label the ~250 that matter (real question / setup / correction / noise) once, and every gate change can be scored against real Indian-English speech instead of invented examples.
- Keep the audio suite.** The 52 TTS files plus the 10 degradation conditions catch STT-prompt regressions (#8) directly — Flask surviving is the acceptance criterion.
- Latency gate in CI.** A run must fail if `since_last_fragment_ms` p50 exceeds ~250 ms or p90 exceeds ~600 ms on the replay corpus.
- Grounding check.** The résumé-grounding script used in §16.1 should run over every benchmark answer set; target is zero ungrounded first-person metrics.
- Sustained-rate test.** 10 questions in 2 minutes against the real provider, measuring the 429 rate — the only way to see F-13 before a real interview does.
- One human read-aloud pass.** Ten answers, read out loud, timed. Anything over ~30 seconds fails.

28. Final Scorecard

Dimension	/10	Evidence
Speech detection (VAD)	8	Adaptive floor, 1 s calibration, 320 ms voiced minimum and 300 ms pre-roll all work; segments were produced correctly for every synthesised utterance including quiet and noisy lines. Lost a point for the 1 s connection-start deafness and for quiet speech failing silently with only a post-hoc counter.
STT accuracy (clean)	8	39/44 synthesised utterances exact. Errors were Django→"D'Ango" and Kubernetes→"Kubernetes".
Technical word recognition	6	Terms in the prompt vocabulary are robust (RAG, Docker: 10/10 degradation conditions). Terms outside it are not (Flask: 8/10 broken). 39 common terms absent; the résumé never reaches the recognizer when a JD is loaded.
Mispronunciation recovery	4	3 of 6 recovered end-to-end. Recovers phonetic nonsense; fails whenever the wrong word is itself a real technology (Rack, doctor, red is). No repair layer exists.
Question segmentation	3	Two of five paused questions split into two answers at every pause length tested; the other two split above ~1.8 s, one of them losing the real question entirely.
Question understanding	6	115/132 on the offline matrix, and every technical mis-transcription reached the gate as a question. Held back by the intent-match truncation (6/15 project questions) and by 8/9 trailing-off fragments being answered.
Follow-up understanding	7	A clean four-turn chain resolved perfectly with the answer style changing each turn; Scenarios A and C passed. Scenario E failed and "Why." with a period is never answered.
Context retention	6	Three turns of history plus résumé/JD are reliably present and used. But context only flows forward: a wrong first answer is written to history and the follow-up resolver actively reinforces it.
Self-correction handling	1	0 of 3 corrections applied, at every pause length. "actually" is stripped as a filler. In production a correction was silently discarded and cost 17 s and three spoken turns.
Multilingual handling	3	Hinglish with an English question word and a "?" is handled well. Devanagari output (1.2 % of real transcripts) is unreadable by the gate and ignored; the one traced instance cost 40 s.
LLM understanding	6	Strong on well-formed input, including informal grammar and multi-part questions. Answers real-word mis-transcriptions literally and echoes the wrong term into the spoken answer.
Answer correctness	6	Technically sound and appropriately specific on clean transcripts (the disk-full and race-condition answers were genuinely good). Undermined by fabricated first-person metrics.
Answer naturalness	5	Short definitional answers score 9/10. Follow-ups run to 107 words at p50 with 5 uniform bullets — 55 % exceed the prompt's own ceiling; 12/18 single-part questions got bullets they should not have.
Human speakability	5	Sentence length p50 = 27 words, max 41. 88 non-ASCII dashes and 25 smart quotes across 33 answers. Inline backticks in code answers.
Latency	8	182 ms p50 last-word→LLM, 598 ms p50 to first token, ≈1.2 s perceived. Genuinely good. Loses points only for the throttling cliff (6–15 s) and the 1.5 s soft wait.
Interruption handling	6	Real barge-in works and the follow-up-await race is correctly handled with a documented owner check. But <code>keyword_match</code> lets an incidental term mention cancel a good answer.
Error recovery	3	<code>close_utterance</code> , the empty-answer path and the error broadcasts are well built. But nothing recovers a wrong answer, a dropped correction, or a deleted fragment — and there is no configured STT fallback.
Reliability	7	No crashes across ~150 pipeline runs in this audit. The decision loop's catch-all, task cancellation and buffer reset are carefully done. Only failure seen was provider 429s.
Observability	9	The strongest dimension. Durable JSONL, turn ids, <code>since_last_fragment_ms</code> , <code>pipeline_ms</code> , first-token latency, STT latency, gate reason and focus, dropped-transcript reasons, sub-threshold counters. This audit was possible <i>because</i> of it.

Dimension	/10	Evidence
Production readiness	5	Deployable and observable, with real capacity limits (4–5 questions/min) and no STT redundancy. The blockers are behavioural, not operational.
Overall	5.6	Fast, well-instrumented and correct on fluent speech; unreliable on the disfluent speech real interviews are made of.

29. Recommended Implementation Roadmap

Ordered by value per unit of regression risk, not by severity. Nothing here should start before the baselines in §27 exist.

Phase 0 — Safety net (before touching any behaviour)

- Freeze golden outputs for the two existing harnesses.
- Adopt Tests A, A2 and E as a permanent offline suite (they run in seconds, need no network).
- Label ~250 utterances from the real corpus as a replay set.
- Record today's latency percentiles as the number every later change is measured against.

Phase 1 — Low risk, immediate value

- **#5** Strip all terminal punctuation before follow-up matching; add the missing phrases. (F-08, F-09, F-22)
- **#4** Forbid an intent match from truncating a content lead-in. (F-05 — fixes a whole family of "generic answer" complaints)
- **#10** `keyword_match` may start but never cancel an answer. (F-14)
- **#6** Do not split clauses on a dot after a single letter or digit. (F-10)
- **#11** Surface throttling and low confidence in the overlay. (F-13, F-03 — makes existing failures visible)
- Set `STT_FALLBACK_ENABLED=false` or supply a key. (F-18)

Gate to Phase 2: full matrix re-run with zero focus-column regressions, and latency percentiles unchanged.

Phase 2 — The core fix

- **#1** Continuation window + merge-and-supersede. (F-01, F-06 — the highest-value change in this report and the one most likely to cost latency)
- **#3** Correction markers, built on top of #1. (F-02)

Gate to Phase 3: Test E shows 1 answer at every pause length for all five patterns; the Flask/Django correction resolves to Django; `since_last_fragment_ms` $p50 \leq 250$ ms.

Phase 3 — Transcript repair

- **#8** Rebuild the STT prompt from extracted résumé/JD terms. (F-11 — do this first; it reduces how much #2 has to catch)
- **#2** Term-normalisation layer, gated on session vocabulary, every substitution logged. (F-03)
- **#7** Duration-aware confidence drop. (F-07)

Gate to Phase 4: the §20 matrix passes for terms present in the session vocabulary; the negative suite (Ruby/Rack, medical) shows zero false substitutions; the noise-only corpus shows no rise in invented transcripts.

Phase 4 — Answer quality

- **#9** Resolve the fabrication/depth prompt conflict; hard word ceiling for follow-ups; mechanical bullets-vs-sentences rule; normalise output punctuation to ASCII. (F-04, F-12)

Gate: zero ungrounded first-person metrics over 30 answers; median answer under 90 words; the prompt's own judged benchmark within its ± 1.8 -point noise band.

Phase 5 — Long tail

F-15 credible question marks · F-16 mid-clause prefix guard · F-17 Devanagari handling · F-19 force-cut grouping · F-20 provisional history on cancel · F-21 empty-answer retry · F-23/F-24 dead-rule cleanup and tier-2 value review.

30. The Final Question

"If a real human speaks naturally during a technical interview — including pauses, fillers, imperfect pronunciation, technical terminology, follow-up questions, corrections, and occasional STT mistakes — can this application still understand the intended question and provide a useful, natural answer?"

PARTIALLY

Yes, reliably, when the interviewer speaks in complete sentences without stopping. That case is genuinely well served: the question is heard, gated, and answered in about 1.2 seconds; fillers are stripped; informal and Hinglish grammar is handled; multi-part questions keep all their parts; a chain of pronoun follow-ups resolves correctly and the answer style changes appropriately at each turn; Whisper with the configured vocabulary held "RAG" intact through every audio degradation tested. On that input the system is fast and good.

No, not reliably, the moment the sentence is interrupted — and real interviewers interrupt themselves constantly. Three specific behaviours account for nearly every failure found:

1. **The system decides a question is finished before the human has finished it.** "Can you explain... RAG?" produced two separate answers at every pause length from 0.2 s to 4.0 s — the split is unconditional. Anything ending in a question mark, and anything opening with a follow-up phrase, is released instantly with no grace period. A two-second thinking pause can also cause the real question to be discarded outright.
2. **A correction is never applied.** "Tell me about your Flask project. Actually, I mean my Django project." answered Flask, at every pause length, and never answered Django. In a real session, "I said the HTTPS methods." — the interviewer explicitly correcting a mishearing — was thrown away, and it took three spoken turns and 17 seconds to get one answer. The words that mark a correction are currently classified as filler noise.
3. **A mis-transcribed term is answered literally when the wrong word happens to be a real word.** "What is Rack?" produced a confident explanation of Ruby Rack middleware — with a résumé and a job description that both name RAG — and the follow-up "Why would you use it?" continued into Nginx versus Puma. This is the important one, because it is *invisible*: the answer on screen is fluent, detailed and completely wrong.

There is one further problem that is not about understanding at all but is arguably the most consequential for the person using this: **the answers invent facts about the candidate.** Every quantified claim measured — 28 of 28 — was absent from the supplied résumé, including two mutually contradictory figures for the same project in the same session, and in production an entirely invented health-tech client. The system prompt forbids this and calls it "the one hard line"; a competing rule demanding a number in every bullet defeats it.

The structural conclusion. The pipeline has a VAD, an STT, a filter, a gate and an LLM, and each of those does its own job competently. What it does not have is any stage whose job is to *repair* — to notice that a fragment is half a sentence, that a correction has replaced a question, or that a word is wrong for this speaker's vocabulary. That absence is why the failures are not scattered: they are all the same missing capability, seen from different angles. The good news is that this makes the work tractable — the fixes in Phases 2 and 3 of the roadmap address all three headline failures, and the latency budget that has been carefully earned is the main thing that must be protected while doing it.

Audit integrity note. No application file was modified during this audit. All test harnesses were written to a scratch directory outside the repository. Findings marked VERIFIED are reproduced from a named test run or a quoted production log line; frequency figures are counted from the 1,441-transcript real corpus and stated even where they weaken a finding's priority (see F-08). Cells in the matrices that could not be reached are left blank rather than estimated. The synthesised-speech tests use US-English voices only and therefore measure the easy case — §18 states this explicitly and relies on real Indian-English session data for accent conclusions.